

Entrega da Avaliação P2

Engenharia de Software — 2026.1

ICEV Remote Sensing Software

Plataforma de Sensoriamento Remoto para Agricultura de Precisão

Luis Gustavo Olimpio

Lauan Matheus

João Leonardi

João Vinícius Castello

Vinícius Henrique

Lucas Benevinuto

José Melquíades

Maio de 2026

Instituto de Ensino Superior — ICEV

Teresina, PI, Brasil

Disciplina: Engenharia de Software — Avaliação P2

Prazo de entrega: 11/05/2026

Sumário

1	Mapeamento dos Entregáveis Obrigatórios	3
2	Documentação Técnica Consolidada	3
3	Decisões de Design Complementares	4
4	Tecnologias — Justificativa Resumida	5
5	Como Avaliar Esta Entrega	5
5.1	Setup rápido (5 minutos)	6
5.2	Roteiro de demonstração (5 minutos)	6
5.3	Para executar a suíte de testes	6
6	Estrutura Final do Repositório	6

1. Mapeamento dos Entregáveis Obrigatórios

Este documento é o ponto de partida da avaliação. Cada entregável obrigatório da P2 está mapeado para o arquivo correspondente no repositório do projeto.

#	Entregável da P2	Localização
1	Código-fonte do MVP	Repositório completo — diretórios <code>api/</code> , <code>web/</code> , <code>data/</code> com versionamento Git
2	Plano de Testes	<code>docs/p2/08-plano-de-testes.pdf</code>
2	Relatório de Testes	<code>docs/p2/09-relatorio-de-testes.pdf</code>
3	Manual do Usuário	<code>docs/p2/07-manual-usuario.pdf</code>
4	Apresentação e Demonstração	<code>docs/p2/apresentacao-p2.pdf</code>
5	Documentação Técnica	Ver Seção 2 (consolidada)

2. Documentação Técnica Consolidada

A documentação técnica está **distribuída em vários documentos especializados** já produzidos no Sprint inicial do projeto (TP1). O escopo de cada um é apresentado na Tabela 1.

Item exigido pela P2	Documento(s)
Descrição detalhada da arquitetura	<code>docs/04-diagramas-arquitetura.md</code> — 8 diagramas (alto nível, componentes, deployment, sequência NDVI, sequência classificação, estrutura de diretórios, fluxo de dados)
Tecnologias utilizadas	<code>docs/04-diagramas-arquitetura.md</code> §8 — stack consolidada frontend + backend
Decisões de design e justificativas	<code>docs/04-diagramas-arquitetura.md</code> §9 (DA-01 a DA-05) + complemento na Seção 3 deste documento (DA-06 a DA-11)
Requisitos funcionais e não funcionais	<code>docs/03-especificacao-requisitos-SRS.md</code> — 20 RFs e 8 RNFs
Backlog priorizado e rastreabilidade	<code>docs/02-backlog-produto.md</code> — 33 histórias em 6 épicos
Visão de produto e usuários-alvo	<code>docs/01-documento-visao-produto.md</code>
Protótipo de interface	<code>docs/05-prototipo-interface.md</code>
Plano de projeto	<code>docs/06-plano-projeto-inicial.md</code>

Tabela 1: Mapeamento dos itens da documentação técnica

Os documentos do TP1 também estão consolidados em docs/tp1/documento-unificado.pdf, gerado a partir de docs/tp1/documento-unificado.tex.

3. Decisões de Design Complementares

Em complemento às decisões DA-01 a DA-05 já documentadas em docs/04-diagramas-arquitetura.md §9, estas são as decisões adicionais relevantes para a entrega do MVP.

DA-06 — Separação `SentinelProcessor` ↔ Camada de Rotas

Decisão. A classe `SentinelProcessor` (em `api/sentinel_processor.py`) concentra toda a lógica de I/O geoespacial e cálculo numérico. As rotas FastAPI em `api/main.py` apenas orquestram e serializam.

Justificativa. Permite testar a camada de processamento em isolamento sem precisar subir o servidor HTTP — exatamente o que fazemos em `tests/test_sentinel_processor.py`. Reduz acoplamento e melhora a manutenibilidade (RNF-05).

DA-07 — Tipagem Estática End-to-End

Decisão. TypeScript estrito no frontend e Pydantic no backend, com modelos espelhados (`Coordinate`, `IndexResult`, `ClassificationResult`).

Justificativa. Erros de contrato entre cliente e servidor são detectados em build/runtime, não em produção. O Pydantic também valida automaticamente a entrada do usuário, eliminando código manual de validação.

DA-08 — Resolução do Produto Sentinel-2 mais Recente como Padrão

Decisão. Quando `product_name` é omitido, o backend ordena os produtos `.SAFE` por nome (que contém data ISO) e usa o mais recente.

Justificativa. Otimiza o caminho feliz do usuário leigo, que normalmente quer “ver agora”. O usuário avançado ainda pode passar um produto específico para comparações temporais.

DA-09 — Suavização Gaussiana com Preservação de NaN

Decisão. O filtro gaussiano (`apply_smooth_filter`) divide o resultado pela máscara também filtrada, preservando o limite do polígono do talhão.

Justificativa. Aplicar gaussiano direto “sangra” os valores válidos para fora do talhão, criando halos artificiais. A correção por máscara é a abordagem padrão em sensoriamento remoto.

DA-10 — Visualização 3D via Downsampling

Decisão. Para o terreno 3D (`web/components/terrain-3d-viewer.tsx`), reduz-se a resolução por fator 4 antes de enviar ao cliente.

Justificativa. Talhões grandes podem ter milhões de pixels. Renderizar isso direto em WebGL trava navegadores em GPU integrada (RNF-01). O `downsample` mantém a interpretação visual sem custo de performance.

DA-11 — Gerenciamento de Dependências Python via uv

Decisão. Usar `uv` (`pyproject.toml` + `uv.lock`) em vez de `pip` + `requirements.txt`.

Justificativa. O `uv` é cerca de dez vezes mais rápido em instalação, garante builds reproduzíveis via *lock file* e simplifica o setup em máquinas novas (`uv sync` resolve tudo). *Trade-off*: dependência extra que o usuário precisa instalar — mitigado pela documentação clara no `README.md`.

4. Tecnologias — Justificativa Resumida

A tabela completa está em `docs/04-diagramas-arquitetura.md` §8. As justificativas-chave estão na Tabela 2.

Tecnologia	Por que esta e não outra
FastAPI	Validação automática via Pydantic, OpenAPI grátis (<code>/docs</code>), <i>async</i> nativo. Alternativas (Flask) exigiriam plugins para o mesmo nível.
rasterio	Padrão de fato para I/O de imagens geoespaciais em Python. Wrapper Pythonic sobre GDAL.
Next.js + React	App Router permite SSR + API routes no mesmo projeto (ex.: <code>/api/fields</code> lê KML do disco sem expor o filesystem).
Leaflet	Maduro, leve, integra bem com tiles do Esri/OSM. Alternativas (Mapbox GL) exigem token e são pagas.
Three.js / react-three-fiber	Visualização 3D nativa em WebGL. Alternativa (Cesium) é <i>overkill</i> para terreno simples.
Tailwind + shadcn/ui	Componentes acessíveis (Radix por baixo), sem CSS-in-JS, sem bundler extra.
fast-xml-parser	KML é XML; este parser é o mais rápido em Node.js sem dependências nativas.

Tabela 2: Justificativas de escolha de tecnologias

5. Como Avaliar Esta Entrega

5.1 Setup rápido (5 minutos)

```
# Pre-requisitos: Python 3.12+, Node 18+, GDAL (brew install gdal)
uv sync                                # instala dependencias Python
cd web && npm install                  # instala dependencias Node
cd ..

# Em 2 terminais:
./run_api.sh                          # backend na porta 8000
cd web && npm run dev                  # frontend na porta 3000
```

5.2 Roteiro de demonstração (5 minutos)

1. Abrir <http://localhost:3000>
2. Selecionar talhão crop field 1
3. Calcular NDVI → ver overlay no mapa, estatísticas e histograma
4. Trocar para EVI → comparar
5. Ativar visualização 3D → rotacionar terreno
6. Ir em /classification → rodar K-Means com 3 classes → ver áreas em hectares

5.3 Para executar a suíte de testes

```
uv sync --group dev                    # instala pytest e dependencias de teste
uv run pytest tests/ -v                # roda 28 testes - ver relatorio em docs
/09
```

6. Estrutura Final do Repositório

```
icev-remote-sensing-software/
|-- README.md                        # Setup + instrucoes de execucao
|-- pyproject.toml                  # Dependencias Python
|-- api/                            # Backend FastAPI
|   |-- main.py
|   '-- sentinel_processor.py
|-- web/                            # Frontend Next.js
|   |-- app/
|   |-- components/
|   |-- lib/
|   '-- types/
|-- data/
|   |-- products/                  # Imagens Sentinel-2 .SAFE
|   '-- KML Fields/                # Poligonos de talhoes
|-- tests/                          # Suite de testes (pytest) [NOVO P2]
```

```
| |-- conftest.py
| |-- test_sentinel_processor.py
| '-- test_api_endpoints.py
'-- docs/
    |-- README.md                # Indice geral
    |-- 01-documento-visao-produto.md    # TP1 - referenciados pela
P2
    |-- 02-backlog-produto.md
    |-- 03-especificacao-requisitos-SRS.md
    |-- 04-diagramas-arquitetura.md
    |-- 05-prototipo-interface.md
    |-- 06-plano-projeto-inicial.md
    |-- p2/                        # [NOVO] Entrega P2
    | |-- 00-ENTREGA-P2.{tex,pdf}    # <- Voce esta aqui
    | |-- 07-manual-usuario.{tex,pdf}
    | |-- 08-plano-de-testes.{tex,pdf}
    | |-- 09-relatorio-de-testes.{tex,pdf}
    | |-- apresentacao-p2.{tex,pdf,pptx}
    | |-- p2-preamble.tex
    | |-- Makefile
    | |-- build_slides.py
    |-- md-sources/                # Originais markdown
'-- tp1/                            # Material complementar do TP1
    |-- documento-unificado.{tex,pdf}
    |-- canvas-experimento.tex
    |-- canvas-experimentofinal.pdf
    |-- atividade-unidade4.{tex,pdf}
    '-- images/
```